Slack autotests — setup and run

This section describes how to run the `test_*_slack_mock.py` suite against a local Omega container that talks to the real Slack Web API. Two Slack bots are used: an “agent” bot, which Omega runs as inside the container, and a “driver” bot, which the pytest harness uses to post prompts into a shared channel and to read the agent’s replies. The LLM is still mocked (`provider="Test"`, deterministic answers from `Autotests/mock/llm.py`); only the message-delivery transport differs from `Autotests/mock/`.

The 26 tests in this directory mirror `Autotests/mock/test_*_mock.py` 1:1: same mock-LLM answers, same prompts, same assertions. They are listed at the end of this document.

1. Prerequisites

- Docker engine on the host.
- Repository checked out, working from its root.
- Python virtual environment under `Autotests/venv` with `pytest` installed.
- A Slack workspace where you can create two apps. The free tier is enough.
- A shared Slack channel in that workspace (public is simplest), where both bots will be members.

2. Configure Slack apps (from scratch)

The suite needs two Slack apps in the same workspace and a shared channel where both of their bot users are members. Walk-through below assumes nothing pre-existing.

2.1. Workspace (skip if you already have one)

A free Slack workspace is enough. Create one at slack.com/get-started if needed.

2.2. Shared channel

In the Slack client, click `+` next to **Channels** in the sidebar, choose **Create channel**, give it a name (for example `omega-qa`), keep it public, create. A private channel works too, but requires extra scopes on both apps (see 2.4).

2.3. Create the two apps

For each role (agent and driver) repeat the same flow at api.slack.com/apps, signed in with the same account that owns the workspace:

1. Click **Create New App**, choose **From scratch**.
2. Set **App Name** (for example `Omega Agent` for the first app and `Omega Driver` for the second; the names are arbitrary, only the resulting tokens matter).
3. Pick the workspace from 2.1.
4. Click **Create App**. You land on the **Basic Information** page of the new app.

2.4. Bot Token Scopes (per app)

For each of the two apps:

1. Open **Features → OAuth & Permissions** in the left sidebar.
2. Scroll to the **Scopes** section, sub-section **Bot Token Scopes** (do NOT use User Token Scopes).
3. Click **Add an OAuth Scope** and add each of the following:
   - `channels:history`
   - `channels:read`
   - `chat:write`
   - `users:read`
4. If the channel from 2.2 is private, also add `groups:history` and `groups:read`.

2.5. Install each app to the workspace

For each of the two apps:

1. Still on **OAuth & Permissions**, scroll to the top section **OAuth Tokens**.
2. Click **Install to Workspace** (or **Reinstall to Workspace** if you added scopes after a previous install).
3. Slack shows a permissions confirmation page; click **Allow**.
4. The page now shows a **Bot User OAuth Token** of the form `xoxb-...`. Copy it.
    - The agent app's token is the value of `SL_BOT_TOKEN` used by the container in step 4 of this README.
    - The driver app's token is the value of `SL_DRIVER_TOKEN` used by pytest in step 5.

2.6. Invite both bots to the shared channel

In the Slack client, inside the channel from 2.2, type:

```
/invite @<agent-bot-username>
/invite @<driver-bot-username>
```

Each bot's username is the one Slack assigned when the app was installed (visible on the app's **Basic Information** page under **Display Information**, and in the workspace member list). Slack may show an "Add this app to the channel?" prompt; confirm. Both bots must show up in the channel's member list after this step.

2.7. Channel id

You need the channel id (`C0...`) of the shared channel. Two ways:

- Slack UI: open the channel, click the channel name at the top to open details, scroll to the bottom of the side panel, the **Channel ID** is at the very bottom.
- API: `curl -s -H "Authorization: Bearer <SL_BOT_TOKEN>" https://slack.com/api/conversations.list` and find the entry by `name`; the `id` field is the channel id.

This value is `SL_CHANNEL_ID`, used both by the container (step 4) and by pytest (step 5).

2.8. Agent bot user id

You also need the bot user id (`U0...`) of the **agent** bot (not the driver). The driver harness filters incoming messages by author so it only treats messages from this user id as agent replies; without it the driver would mistake its own messages for the agent's.

The user id is the `user_id` field returned by `auth.test` called with the agent token:

```
curl -s -H "Authorization: Bearer <SL_BOT_TOKEN>" \
  https://slack.com/api/auth.test | python -m json.tool
```

This value is `SL_AGENT_USER_ID`, used by pytest only (step 5).

2.9. Summary of what you should have collected

Value	Source	Used by
Agent `xoxb-...` token	Agent app, **OAuth & Permissions → Bot User OAuth Token**	container, as `SL_BOT_TOKEN`
Driver `xoxb-...` token	Driver app, **OAuth & Permissions → Bot User OAuth Token**	pytest, as `SL_DRIVER_TOKEN`
Channel id `C0...`	Slack UI or `conversations.list`	container and pytest, as `SL_CHANNEL_ID`
Agent user id `U0...`	`auth.test` with the agent token	pytest, as `SL_AGENT_USER_ID`

3. Build the local image

The mock infrastructure is part of the source tree, so the image must be built locally rather than pulled from the registry.

```
docker build -t omega:mock .
```

4. Start the container with the Test provider and the Slack channel

Use the `scripts/omega` wrapper. It takes care of `--init`, `--user`, the `--tmpfs` mounts, `--security-opt no-new-privileges`, the persistent memory volume, and the `commchannel`/`provider`/`embeddingprovider` arguments, so the test setup stays in sync with how the agent is started in production and in CI.

The container connects back to the host on TCP port 9765 to reach the mock LLM controller. `TEST_SERVER_IP` must hold the host IP that is reachable from inside the container; under the default Docker bridge this is `172.17.0.1`. The Slack adapter inside the container talks directly to `slack.com/api`.

```
env TEST_SERVER_IP=172.17.0.1 \
    SL_BOT_TOKEN="<agent_bot_token>" \
    SL_CHANNEL_ID="<channel_id>" \
    ./scripts/omega start -s 0000 -p Test -t slack -d omega:mock
```

Notes:

- `-t slack` selects the Slack adapter inside `src/channels.metta`.
- `-p Test` selects the mock LLM dispatcher.
- `-s 0000` sets `OMEGA_AUTH_SECRET` inside the container; the test harness sends this same secret via the driver bot during `_sl_authenticate` once per session, binding the driver as the agent's owner.
- `SL_BOT_TOKEN` is the agent bot token (the bot Omega runs as).
- `SL_CHANNEL_ID` is the shared channel both bots live in.
- `TEST_SERVER_IP=172.17.0.1` is the host's docker-bridge address used by the mock LLM provider. It must be set even for the Slack channel, because `provider=Test` reads it.

Wait until the agent loop is up. The first runtime `CHARS_SENT:` line (with a byte count after the colon) marks the end of `initChannels` / `initMemory`:

```
until docker logs omega 2>&1 | grep -qE "CHARS_SENT: [0-9]+"; do sleep 2; done
```

5. Configure the test environment

Export the variables the test harness reads.

```
export OMEGA_CONTAINER=omega
export OMEGA_AUTH_SECRET=0000
export SL_DRIVER_TOKEN="<driver_bot_token>"
export SL_CHANNEL_ID="<channel_id>"
export SL_AGENT_USER_ID="<agent_bot_user_id>"
export OMEGA_GIT_TOKEN=<github_pat>     # only required by test_git_push_to_remote_slack_mock
```

Variable	Required	Description
`OMEGA_CONTAINER`	Yes	Container name passed to `docker exec` from the harness. Must equal the container name from step 4 (`omega` when using the script).
`OMEGA_AUTH_SECRET`	Yes	Auth secret used by the driver bot once per session. Must equal the `-s` value from step 4.
`SL_DRIVER_TOKEN`	Yes	Driver bot token. Tests are skipped if unset.
`SL_CHANNEL_ID`	Yes	Shared channel id. Same value the container was started with.
`SL_AGENT_USER_ID`	Yes	Bot user id of the agent app. The driver bot filters incoming messages by author and ignores everything except messages from this user id.

`OMEGA_GIT_TOKEN`	No	GitHub PAT used by `test_git_push_to_remote_slack_mock`. The test is skipped if this variable is unset.

6. Run the suite

```
cd Autotests
source venv/bin/activate
pytest -s -v mock_slack/test_*_slack_mock.py
```

The `LlmMockController` and the `SlackRealDriver` are provided by session-scoped fixtures in `mock_slack/conftest.py`, so both are started once per pytest session. Expected output: `26 passed` (or `25 passed, 1 skipped` if `OMEGA_GIT_TOKEN` is not set).

7. Tear down

```
./scripts/omega clean
```

This removes the `omega` container and the `omega-memory` volume created by the script in step 4.

Tests description

All 26 tests are 1:1 mirrors of the corresponding `Autotests/mock/test_*_mock.py` files. The mock-LLM answer, prompt body, prepared fixtures, and assertions are identical to the comm-channel variants; the only difference is the message-delivery transport. Where the comm-channel variant calls `comm.send_message(prompt)`, the Slack variant calls `sl_send_prompt(sl, prompt)`, which makes the driver bot post `chat.postMessage` into the shared channel. Because the LLM is deterministic, no `try_with_clarification` retries are needed; every test either passes on the first attempt or fails outright.

Creating files

1. test_create_file_slack_mock.py

Creates `/tmp/testcat/hello.txt` containing exactly `Hello`.

- Mock answer: `(shell "mkdir -p /tmp/testcat") (write-file "/tmp/testcat/hello.txt" "Hello")`.
- Checks: directory exists, file exists, permissions start with `-rw`, mtime ≥ test start, content is `Hello` (with or without trailing newline).

2. test_create_empty_file_slack_mock.py

Creates an empty `/tmp/test_empty/hello.txt`.

- Mock answer: `(shell "mkdir -p /tmp/test_empty") (write-file "/tmp/test_empty/hello.txt" "")`.
- Checks: file is created, `cat` returns an empty string.

3. test_create_script_slack_mock.py

Creates `/tmp/test_script/date.sh`, a shell script that prints the current date.

- Mock answer: `(shell "mkdir -p /tmp/test_script") (write-file "/tmp/test_script/date.sh" "#!/bin/bash\ndate\n") (shell "chmod +x /tmp/test_script/date.sh")`.
- Checks: file exists, is executable (`x` in permissions), harness runs it via `sh date.sh` and verifies the output contains the current year (UTC or local).

Editing files

4. test_edit_add_timestamp_slack_mock.py

Appends the current timestamp as a new line to a pre-created `note.txt`.

- Mock answer: `(shell "date -Iseconds >> {TARGET_FILE}")`.
- Checks: mtime changed, last line of the file contains ≥ 4 digits.

5. test_edit_delete_line_slack_mock.py

Deletes the second line from a pre-created 3-line file.

- Mock answer: `(shell "sed -i 2d {TARGET_FILE}")`.
- Checks: mtime changed, exactly 2 lines remain, lines 1 and 3 intact.

6. test_edit_append_line_slack_mock.py

Appends a 4th line `Delta` to a pre-created 3-line file (Alpha / Bravo / Charlie).

- Mock answer: `(shell "printf '%s\\n' Delta >> {TARGET_FILE}")`.
- Checks: mtime changed, 4 lines total, first 3 unchanged, 4th line equals `Delta`.

7. test_convert_format_slack_mock.py

Converts `document.md` → `document.txt`, preserving textual content.

- Mock answer: `(shell "cp {SOURCE_FILE} {DEST_FILE}")`.
- Checks: `.txt` exists, contains the keywords `My Title`, `Some paragraph text`, `item one`, `item two`.

Running shell scripts

8. test_run_error_script_slack_mock.py

Runs a syntactically broken pre-created script and captures stdout and stderr to a file.

- Mock answer: `(shell "sh {SCRIPT_FILE} > {OUTPUT_FILE} 2>&1")`.
- Checks: `output.txt` exists, contains the literal `start` (stdout) AND at least one of `error` / `syntax` / `unexpected` / `missing` / `not found` (stderr); container is still alive.

9. test_run_repeated_slack_mock.py

Runs `dateupdate.sh` exactly 10 times in a row.

- Mock answer: ten consecutive `(shell "{SCRIPT_FILE}")` calls (one per run).
- Checks: `update.txt` exists with mtime ≥ start, has ≥ 10 lines, every line contains date-like digits.

Internet search

10. test_search_basic_slack_mock.py

Sends prompt “What is SingularityNet? Search the web.”

- Mock answer: `(send "SingularityNet (SNET) is a decentralized AI marketplace founded by Ben Goertzel. Its native token AGIX powers a network of AI agents and services, and it is part of the broader ASI Alliance ecosystem.")`.
- Checks: the reply contains at least one of `singularitynet`, `agi`, `blockchain`, `decentralized`, `marketplace`, `goertzel`.

11. test_search_weather_slack_mock.py

Returns a synthetic Valencia weather reply (the live variant cross-checks against open-meteo; the mock variant fixes a synthetic reference `REF_TEMP_C = 18.0` and stays fully offline).

- Mock answer: `(send "Current weather in Valencia, Spain: about 18.0°C.")`.
- Checks: the reply contains a plausible Celsius number (range [-20; 50]); at least one of those numbers is within ±10 °C of the synthetic reference 18.0 °C.

12. test_search_invalid_slack_mock.py

Asks about a gibberish string.

- Mock answer: `(send "No results found for <gibberish>. The string appears to be gibberish, no meaningful matches.")`.
- Checks: the reply contains a negation phrase (`no results`, `not found`, `gibberish`, `nonsense`, `no meaning`, `unknown`, …).

Memory

13. test_memory_chromadb_slack_mock.py

Requests the agent to remember a fact tagged with marker `CI-SMOKE-<run_id>`.

- Mock answer: `(remember "Unique smoke marker CI-SMOKE-<run_id> was emitted by CI.")`.
- Checks: `(remember ...)` was invoked with the marker; vector count in the `embeddings` table of `chroma.sqlite3` grew by ≥ 1.

14. test_memory_history_slack_mock.py

Sends “Acknowledge with one short line that you received marker `<run_id>`.” and verifies the entry in `history.metta`.

- Mock answer: `(send "Acknowledged marker <run_id>.")`.
- Checks: an s-exp record referencing `REQ-<run_id>` appears in history; the agent issued `(send ...)`; file mtime and size grew.

Skills

15. test_skill_metta_slack_mock.py

Asks the agent to evaluate a short MeTTa expression and report the result.

- Mock answer: `(metta "(+ 2 2)") (send "The metta skill evaluated (+ 2 2) and returned 4.")`.
- Checks: `(metta ...)` was invoked; the agent then issued a `(send ...)`. Semantic correctness of the MeTTa expression is not checked; the goal is to exercise the skill.

16. test_skill_pin_slack_mock.py

Gives a multi-step task (“restarting servers alpha → beta → gamma, just finished alpha”) and expects the agent to track progress with `pin`.

- Mock answer: `(pin "Server restart progress: alpha done; beta and gamma pending.") (send "Tracking: alpha done, beta and gamma pending.")`.
- Checks: `(pin ...)` was invoked whose argument references either the `run_id` or one of the keywords `step` / `alpha` / `beta` / `gamma` / `restart` / `server` / `done`; the agent acknowledged via `(send ...)`.

Working with git

17. test_git_pull_public_slack_mock.py

Agent clones a public repository over anonymous HTTPS, no token.

- Mock answer: `(shell "rm -rf {TARGET_DIR} && git clone {remote} {TARGET_DIR}")`.
- Checks: `.git/` appears, HEAD points to a real commit, ≥ 1 tracked file in HEAD, origin matches the expected remote URL (normalized, trailing `/` and `.git` ignored).

18. test_git_local_commit_slack_mock.py

Agent runs `git init`, `git add`, `git commit` locally inside the container.

- Mock answer: chain of `(shell "git -C {TARGET_DIR} init") (shell "...write file...") (shell "git -C {TARGET_DIR} add -A") (shell "git -C {TARGET_DIR} commit -m 'add hello <run_id>'")`.
- Checks: HEAD has at least one commit, commit subject contains the `run_id` (warning, not failure), the file is

present in the tree.

19. test_git_push_to_remote_slack_mock.py

Agent clones a remote, creates branch `qa/run-<id>`, adds a file, commits, and pushes.

- Mock answer: single `(shell "rm -rf ... && git clone ... && cd ... && git checkout -b ... && printf ... > <file> && git add -A && git commit -m '...' && git push -u origin ...")`.
- Parameters via env vars: `OMEGA_GIT_TOKEN` (token; never appears in code) and `OMEGA_GIT_REMOTE` (default `https://github.com/OmegaSing/Test-Repopo`). Test is skipped if the token variable is unset.
- Checks: branch present on remote (GitHub API 200), file present on branch, the shell call included `git push`, credentials wiped on teardown.

Multi-skill tests

20. test_run_create_dirs_slack_mock.py

Agent writes `mkdirs.sh` and runs it. The script must create `test1`, `test2`, `test3` inside `/tmp/test_dirs/`.

- Mock answer: `(write-file "{SCRIPT_PATH}" "#!/bin/bash\nmkdir -p .../test1 .../test2 .../test3\n") (shell "chmod +x {SCRIPT_PATH}") (shell "{SCRIPT_PATH}")`.
- Checks: all three directories exist with fresh mtimes; agent invoked `(write-file ...)` referencing `mkdirs.sh`; agent invoked `(shell ...)` to run the script. Diagnostics print `wf=<count>, sh=<count>, perms=<...>` to make stalls obvious.

21. test_memory_episode_slack_mock.py

Two-turn flow: tells the agent that the user's dog Barney lost a baby tooth, waits 5 seconds, then asks to recall when this happened.

- Turn 1 mock answer: `(remember "Barney the dog lost his first baby tooth at the vet today.")`.
- Turn 2 mock answer: `(query "Barney tooth") (send "Barney the dog lost his first baby tooth on <YYYY-MM-DD>. The milestone is recorded in my notes.")`.
- Checks (turn 1): `(remember ...)` was invoked whose argument contains `tooth` or `Barney`. Checks (turn 2): `(query ...)` or `(episodes ...)` was invoked; the reply contains at least one of `dog` / `tooth` / `lost`; the reply contains the captured seed date in `YYYY-MM-DD` format.

22. test_skill_query_slack_mock.py

Two-turn flow: plant a unique color (`azure-<run_id>`) via `remember`, wait for embeddings to settle, then ask the agent to recall it via `query` (embedding lookup, not timestamp lookup).

- Turn 1 mock answer: `(remember "My favorite color is azure-<run_id>.") (send "Stored: favorite colour is azure-<run_id>.")`.
- Turn 2 mock answer: `(query "favorite color") (send "Your favorite color is azure-<run_id>.")`.
- Checks (turn 1): `(remember ...)` carried the secret color. Checks (turn 2): `(query ...)` was invoked; the reply mentions the secret color verbatim.

23. test_skill_episodes_slack_mock.py

Two-turn flow: send a message tagged with a unique keyword (no `remember`), capture the timestamp, then ask the agent to use `episodes` (timestamp lookup, not `query`) to recall what was discussed at that earlier time.

- Turn 1 mock answer: `(send "Acknowledged keyword <marker>.")`.
- Turn 2 mock answer: `(episodes "<seed_ts>") (send "The unique keyword was <marker>.")`.
- Checks (turn 1): the turn is recorded in `history.metta` with a timestamp. Checks (turn 2): `(episodes ...)` was invoked for the seed timestamp; the reply mentions the original marker.

24. test_complex_weather_flow_slack_mock.py

Four-step pipeline: search NY weather → write `w.txt` with the forecast → write `p.sh` extracting the first Celsius number into `t.txt` → run `p.sh`. Because the mock controls only the LLM dispatch (the network-bound `search` skill is not exercised), the mocked response provides the forecast text directly.

- Mock answer: `(write-file "/tmp/wflow/w.txt" "New York tomorrow: clear, high 22 degrees Celsius.") (write-file "/tmp/wflow/p.sh" "#!/bin/bash\ngrep -oE '[0-9]+' /tmp/wflow/w.txt | head -1 > /tmp/wflow/t.txt\n") (shell "chmod +x /tmp/wflow/p.sh") (shell "/tmp/wflow/p.sh")`.
- Checks: `w.txt` exists; history contains `(write-file ...)` referencing `w.txt`; `p.sh` exists with executable bit; history contains `(write-file ...)` or `(shell ...)` referencing `p.sh`; `t.txt` exists; history contains `(shell ...)` running `p.sh`; `t.txt` content is a number in the range [-60; 120]; content length ≤ 40 characters.